Institute for Complex Systems

JOHANNES KEPLER UNIVERSITÄT LINZ

Univ.-Prof. Dr. Daniel Große
DI Simon Reitinger & Katharina Ruep, M.Sc.

16. April 2026

Übung 4
zur Vorlesung Rechnerarchitektur

Abgabe bis **Donnerstag 30 April, 2026 10:15** via EPICS: `https://ep.ics.jku.at`.

Für VHDL Aufgaben stehen in Moodle Templates zur Verfügung. Diese enthalten die *entity* (Schnittstelle) der zu realisierenden Schaltung und eine dazu passende *testbench*. Die Templates sind mit der *architecture* der zu realisierenden Schaltung (und gegebenenfalls weiteren benötigten Komponenten) zu vervollständigen. Dabei dürfen nur strukturelle Beschreibungen und bitweise Operationen (`AND`, `OR`, `NOT`, ...) verwendet werden. Die resultierenden VHDL Dateien müssen vor der Abgabe in EP.ICS mit *ghdl* ausgeführt werden. Dateien, die nicht kompilieren, können nicht abgegeben werden.

Aufgabe 1 – Countdown (8 Punkte)

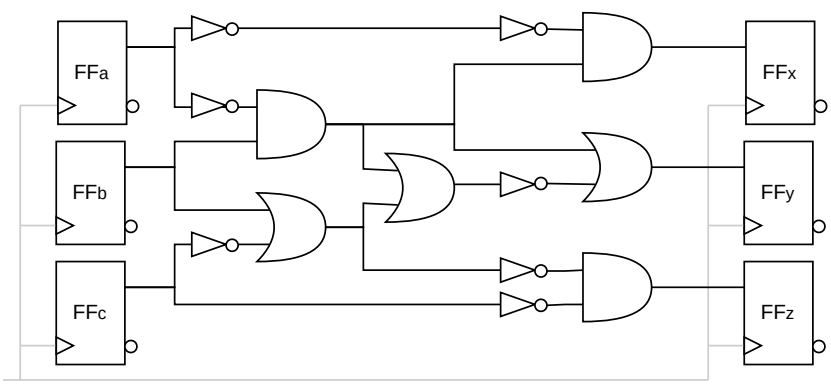
Implementiere einen (sequentiellen) Zähler für positive Zahlen, der zu jeder positiven Flanke von `clk` seinen Wert um 1 vermindert. Dabei soll er bei der größtmöglichen Zahl beginnen und bis 0 zählen. Bei einem Unterlauf (underflow) soll der Zähler wieder vom größtem Wert beginnen und eine 1 für einen Takt bei `alarm` ausgeben. Eine 1 auf dem Reset-Signal soll den Zähler zurücksetzen. Außerdem soll er eine generische Bitbreite N unterstützen. Der Zähler soll dabei Eingänge für Takt (`clk`) und Reset (`reset`) sowie die Ausgänge (`y`), für die aktuelle Zahl und Underflow (`alarm`), um einen Übergang von 0 auf die größtmögliche Zahl haben.

Aufgabe 2 – Multiplizierer (10 Punkte)

Implementiere einen kombinatorischen Multiplizierer in VHDL, welcher zwei positive 16-Bit Zahlen (unsigned) multipliziert und eine positive 32-Bit Zahl (unsigned) als Ergebnis hat. Die 16 Partialprodukte sollen dabei mit Hilfe von Addierern baumartig zusammenfasst werden.
Hinweis: Bitweise Operatoren (`AND`, `OR`, `XOR`, ...) können auf Signale mit beliebiger Breite angewendet werden. Mit dem Ausdruck `(7 downto 0 => k)` kann mehreren Einträgen eines Vectors der Wert des Signals `k` zugewiesen werden. Außerdem können Signale mit Hilfe des Operators `&` konkateniert werden.

Aufgabe 3 – Timing Analyse (6 Punkte)

Gegeben ist folgende Schaltung und Verzögerungszeiten der FFs und Gatter. Alle FFs sind mit dem selben Clock verbunden (in grau).



FF	t_{pcq}	= 40 ps
	t_{ccq}	= 25 ps
	t_{setup}	= 50 ps
	t_{hold}	= 80 ps
gates	t_{pd-and}	= 30 ps
	t_{cd-and}	= 25 ps
	t_{pd-or}	= 35 ps
	t_{cd-or}	= 30 ps
	t_{pd-not}	= 15 ps
	t_{cd-not}	= 10 ps
	t_{pd-buf}	= 10 ps
	t_{cd-buf}	= 5 ps

Berechne

a) den längsten komb. Pfad (t_{pd}) und

b) den kürzesten komb. Pfad (t_{cd})

der Schaltung und markiere diese in der Schaltung.

c) Ermittle den maximalen Takt mit dem das *Setup Time Constraint* noch eingehalten werden kann. *Hinweis: Einheiten nicht vergessen.*

d) Kann das *Hold Time Constraint* eingehalten werden? Falls nicht ändere die Schaltung mithilfe von Buffern und gib die aktualisierten Minimal-Verzögerungen an.

   Füge Buffer so ein, dass die Maximale Verzögerungszeit nicht verändert wird! Es müssen auch andere Pfade berücksichtigt werden; das *Hold Time Constraint* muss auf allen Pfaden eingehalten werden. Achte darauf so wenig Buffer wie möglich einzufügen.